

Introduction

Every so often a "rated easy" LeetCode problem quietly tests something that has nothing to do with difficulty and everything to do with discipline: did you actually read the constraints, or did you pattern-match to the nearest thing you've seen before? LeetCode 2996, "Smallest Missing Integer Greater Than Sequential Prefix Sum," is exactly that kind of problem. It looks like a straightforward "find the missing number" exercise, but it hides a trap that catches people who reach for the obvious shortcut without checking whether the array's assumptions actually hold.

This pattern — numbers that are supposed to be issued in strict sequence, with a search for "the next safe value" once the sequence breaks — shows up constantly outside of interviews too. Auto-incrementing database IDs, invoice numbers, ticket queues, coupon codes: anywhere a system hands out numbers in order and something can create a gap, you need exactly this logic. Get the edge case wrong in production and you either reuse a number that's already spoken for or skip one that should have been assigned. That's not a hypothetical bug, it's the kind of thing that causes duplicate invoice numbers or ID collisions in a live system.

Interviewers like this problem because it rewards precise reading over pattern memorization. You'll walk away from this article knowing exactly how to solve it, why the "obvious" approach fails, and how to spot this family of problems instantly.

Problem Overview

You're given an array of integers, `nums`. Look at the *sequential prefix* — the longest run starting at index 0 where each number is exactly one greater than the number before it. Sum up that prefix. Then find the smallest integer that is greater than or equal to that sum and does **not** appear anywhere in `nums`. Return it.

In plain terms: walk from the start of the array as long as the numbers climb by exactly one each step. Add those numbers together. That sum becomes your starting point for a search — you now need the smallest number, from that point upward, that the array hasn't already claimed.

Example

Take `nums = [1, 2, 3, 2, 5]`.

- The sequential prefix is `1, 2, 3` — each value is exactly one more than the last.
- Sum of that prefix: `1 + 2 + 3 = 6`.
- Is `6` in the array? No. So `6` is the answer.

Now take `nums = [3, 4, 5]`.

- The array is sequential the whole way through (`3, 4, 5`), but it doesn't *start* at `1`.
- Because `nums[0] != 1`, the number `1` itself was never claimed by anything.
- The answer is simply `1` — no summing required at all.

And the trap case, `nums = [2, 2]`:

- `nums[0]` is `2`, not `1`.
- The answer is immediately `1`, before you even look at the rest of the array.

That last one is the case that catches people who don't check the starting value first.

Intuition

The first instinct almost everyone has: take the maximum value in the array and return `max + 1`. It feels safe — surely nothing bigger than everything you've seen could already be "taken."

It's wrong, and the reason is instructive. Consider `[1, 2, 3, 2, 5]` again. The max is `5`, so the naive approach returns `6`. That happens to be correct here — but only by coincidence. The real question was never "what's bigger than everything in the array," it was "what number does the *sequential run at the start* not account for." The max-plus-one trick ignores the actual structure of the problem: gaps like `4` in the middle of the array, or the array simply not starting at `1`, are invisible to it.

So instead of looking at the whole array, look only at how far it stays perfectly sequential from the very beginning. Walk forward while each number is exactly one more than the previous one. The moment that pattern breaks, stop. Sum every number you walked through — that sum represents the total "claimed territory" of the prefix, and the smallest number *not* claimed by that prefix is one more than the sum.

Two things make this work:

1. **Why the sum, and not just `index + 1`?** Because the values in the prefix don't have to be small or contiguous starting from a fixed point once duplicates are involved — the sum is what actually accounts for the total space consumed, and it stays correct even when a duplicate later

breaks the streak. Index-based counting silently drifts wrong the instant the array has any hiccup upstream.

2. **Why check `nums[0]` first?** Because the entire sum-based approach only means something if the prefix started at `1`. If it didn't, `1` itself is unclaimed and always wins — this is the `[3, 4, 5]` and `[2, 2]` trap.

Once the sum is established, you're not done — you still need to confirm the candidate (`sum + 1`) isn't sitting somewhere else in the array, and keep incrementing until you find one that's genuinely missing.

Brute Force Solution

Idea: Build a set of all values in `nums` for $O(1)$ lookups, compute the sequential prefix sum the same way, then linearly search upward from `sum + 1` (or `1`, if the trap case applies) checking set membership.

Algorithm: 1. Convert `nums` to a set. 2. Walk the array to find the sequential prefix sum, same logic as the optimal approach. 3. Starting from the candidate value, increment until a value isn't in the set.

Advantages: Set lookups are $O(1)$, so the search phase is fast per check.

Disadvantages: You pay $O(n)$ extra space to build the set, which is unnecessary for arrays this small — the number of candidates you'll actually check is tiny in practice, so the set buys you speed you don't need at the cost of memory you didn't have to spend.

Time complexity: $O(n)$ to build the set, plus $O(k)$ for the search where `k` is the number of candidates checked. **Space complexity:** $O(n)$ for the set.

Optimal Solution

Skip the set. The array is small enough, and the number of membership checks is small enough, that `value in nums` (a linear scan) doesn't meaningfully change your runtime — the dominant cost is still the initial $O(n)$ pass.

Here's the full plan:

- | Step | Action |
|------|---|
| 1 | If <code>nums[0] != 1</code> , return <code>1</code> immediately — the trap case. |
| 2 | Otherwise, walk the array, summing each value while it continues the run (<code>nums[i] == nums[i-1] + 1</code>). |
| 3 | Stop the moment the run breaks (or the array ends). |

Step**Action**

- 4 Starting at `sum + 1`, increment until you find a value not present in `nums`.
- 5 Return that value.

Walking through `[3, 4, 5, 1, 12, 14, 13]`: `nums[0]` is `3`, not `1`, so the trap fires — wait, that's not right for this array, let's trace it properly. Actually `nums[0] = 3 != 1` ...

Hold on — this specific example needs the full walk to explain correctly, so here it is precisely: the sequential-prefix rule only tracks numbers that increase by exactly one *starting from whatever* `nums[0]` is for the purpose of building the sum, but the "return 1 immediately" shortcut only applies when the problem is asking about the integer `1` specifically. In `[3, 4, 5, 1, 12, 14, 13]`, the prefix `3, 4, 5` is sequential, and the running sum is `12`. Since the prefix didn't start at `1`, the sum itself is still the correct floor — the candidate search begins at `sum + 1 = 13` ... but `13` is in the array, `14` is in the array, and `15` is missing. **Answer: 15.**

Python Code

```
def missing_integer(nums: list[int]) -> int:
    """LeetCode 2996: smallest missing integer >= sequential prefix sum."""
    total = nums[0]

    # Extend the running sum while the prefix keeps climbing by exactly 1
    for i in range(1, len(nums)):
        if nums[i] == nums[i - 1] + 1:
            total += nums[i]
        else:
            break

    # Search upward from total + 1 for the first value missing from nums
    candidate = total + 1
    nums_set = set(nums)
    while candidate in nums_set:
        candidate += 1

    return candidate
```

Code Walkthrough

- `total = nums[0]` seeds the running sum with the first element — the prefix always includes at least this one value.
- The `for` loop walks from index `1` onward. As long as each value is exactly one more than its predecessor, it's added to `total`. The instant that condition fails, `break` stops the walk — the

sequential prefix is over.

- `candidate = total + 1` sets the starting point for the search: the smallest value not yet accounted for by the prefix.
- `nums_set = set(nums)` converts the array once for $O(1)$ membership checks during the search loop — cheap insurance against a pathological array where many consecutive candidates are all present.
- The `while` loop increments `candidate` until it lands on a value that isn't in `nums`, then returns it.

Note this version doesn't special-case `nums[0] != 1` separately — it doesn't need to. If `nums[0]` isn't `1`, the loop still runs correctly, but you can add an early return for clarity and a tiny speed win:

```
def missing_integer(nums: list[int]) -> int:
    if nums[0] != 1:
        return 1

    total = nums[0]
    for i in range(1, len(nums)):
        if nums[i] == nums[i - 1] + 1:
            total += nums[i]
        else:
            break

    candidate = total + 1
    nums_set = set(nums)
    while candidate in nums_set:
        candidate += 1

    return candidate
```

This second version mirrors the trap case explicitly, which makes the intent obvious to the next person reading the code — and is the version worth using in an interview, since it shows you saw the edge case rather than stumbled into handling it by accident.

Dry Run

Trace `nums = [1, 3, 5]`:

1. `nums[0] == 1`, so no early return.
2. `total = 1`.
3. `i = 1`: `nums[1] = 3`. Is `3 == nums[0] + 1 = 2`? No. Break immediately.
4. `total` stays `1`. `candidate = 2`.
5. Is `2` in `{1, 3, 5}`? No. Return `2`.

Trace `nums = [2, 2]` :

1. `nums[0] = 2 != 1` → return `1` immediately. Done.

Complexity Analysis

Time: $O(n)$. The prefix walk touches each element at most once. Building the set is $O(n)$. The candidate search, in the worst realistic case, touches a small, bounded number of values relative to `n` — each array element is inspected a small constant number of times overall.

Space: $O(n)$ for the set used in membership checks; $O(1)$ beyond the input if you skip the set and check membership directly against the list (fine for small `n`, worse asymptotically for large `n` with many collisions).

Both bounds are tight: you cannot avoid at least one full pass over the array (you need every element to know the prefix and to know what's "taken"), and the set is the standard trade of memory for guaranteed $O(1)$ lookups.

Alternative Solutions

You could skip the set entirely and use `candidate in nums` (a list scan) instead. For small inputs, this is honestly the more "lazy" and perfectly acceptable choice — it avoids building a second data structure for a handful of membership checks. It only becomes a real liability if `nums` is large and many consecutive candidates happen to collide with existing values, since each check becomes $O(n)$ instead of $O(1)$.

There's no meaningfully different algorithmic approach here — sorting, heaps, or DP add complexity without solving anything the single pass doesn't already handle.

Edge Cases

- **Array starts above 1** (`[3, 4, 5]`): trap case, return `1` immediately.
- **Duplicate at the very start** (`[2, 2]`): trap case fires before the loop even matters.
- **Fully sequential array with no gaps** (`[1, 2, 3, 4]`): the streak runs the whole array; return the total sum (since nothing broke the streak, the sum itself was never claimed).
- **Single-element array** (`[7]`): if `nums[0] != 1`, return `1`. If `nums[0] == 1`, the prefix is just `1`, and you search from `2`.

- **Large gap immediately after the prefix** (`[1, 2, 3, 100]`): sum is `6`, candidate search finds `6` missing immediately (since only `1, 2, 3, 100` are present).

Common Mistakes

1. **Using** `max(nums) + 1` instead of the sequential prefix sum — ignores gaps entirely.
2. **Forgetting to check** `nums[0] != 1` before trusting the sum — this is the single most common failure mode.
3. **Using** `index + 1` instead of the running sum — breaks silently the moment a duplicate or out-of-order value appears in the prefix.
4. **Off-by-one in the break condition** — comparing `nums[i] == nums[i-1]` instead of `nums[i] == nums[i-1] + 1`.
5. **Not handling a single-element array** — forgetting the loop simply never executes and `total` stays as `nums[0]`.

Interview Questions

- What if the array is sorted but empty — how do you guard against `nums[0]` on an empty list?
- How would your solution change if the array could contain negative numbers?
- Can you solve this without building a `set`, and what's the trade-off?
- What's the smallest input that forces your candidate search loop to run more than once?
- How would you extend this to return *all* missing integers up to some bound, not just the smallest?

Similar Problems

- **LeetCode 41 – First Missing Positive**: same "smallest missing" theme, no sequential-prefix twist, classic $O(n)$ time / $O(1)$ space challenge.
- **LeetCode 268 – Missing Number**: simpler version, single missing number in a known range, good warm-up for the mental model here.
- **LeetCode 1539 – Kth Missing Positive Number**: extends the "find missing values" idea to the k th missing value instead of the first.

Key Takeaways

Walk the sequential prefix from the start, summing as you go, and stop the instant the streak breaks. If the array doesn't start at `1`, the answer is `1` — no computation needed. Otherwise, the answer is the smallest value at or above `sum + 1` that the array hasn't already claimed. The core lesson: don't reach for `max + 1` as a shortcut when the problem is actually about *sequential* structure, and always sanity-check the array's starting assumption before trusting downstream math built on top of it.

Watch the Video

Prefer to see this traced live, including two quiz rounds to test your understanding? {LINK}

About the Series

This walkthrough is part of **Fun with Learning Technology**, a daily series that takes one coding problem and actually breaks it down — not just to get to a working solution, but to understand why it works and where the traps are. New problems drop daily, covering the patterns that show up again and again in real interviews.

Call To Action

If this helped the pattern click, a like on the video goes a long way, and subscribing means you won't miss tomorrow's problem. Subscribe to the Substack for the written breakdown delivered straight to your inbox. Drop a comment with questions, alternative approaches, or requests for problems you'd like to see covered — and share this with anyone prepping for interviews right now.

Quick Review

Pattern: build result by comparing running total vs sorted array?

Greedy prefix-sum scan — when you see 'sum so far' + 'next allowed value', track prefix sum and compare to array, not heap/DP.

Time complexity of the prefix-sum-vs-array scan?

$O(n \log n)$ if sorting needed, $O(n)$ if already sorted — single pass comparing running sum to current element.

Space complexity of this approach?

$O(1)$ extra space — just a running sum variable, no auxiliary structures needed.

Trickiest bug: what happens at the very first element?

Prefix sum starts at 0; must check missing integer could be smaller than `array[0]` before any addition happens.

Why not use a max-heap or DP here?

Overkill — the array's order already encodes the constraint; heap/DP add complexity with no benefit over a linear scan.

Follow-up: how does this pattern show up in real systems?

Coupon/ledger validation — checking a running balance against the next expected entry to catch the first gap.

Smallest Missing Integer Greater Than Sequential Prefix Sum — free Python cheat sheet